Follow the Will of the Market: A Context-Informed Drift-Aware Method for Stock Prediction

Chen-Hui Song
Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
sch21@mails.tsinghua.edu.cn

Xi Xiao*
Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
xiaox@sz.tsinghua.edu.cn

Bin Zhang
Peng Cheng Laboratory
Shenzhen, China
bin.zhang@pcl.ac.cn

Shu-Tao Xia*
Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
Peng Cheng Laboratory
Shenzhen, China
xiast@sz.tsinghua.edu.cn

ABSTRACT

The dynamic nature of stock market styles, referred to as concept drift, poses a formidable challenge when applying deep learning to stock prediction. Models trained on historical data often struggle to adapt to the latest market styles, as the patterns they have learned may no longer hold true over time. To alleviate this issue, the recently popularized concept of In-Context learning has provided us with valuable insights. In this approach, large language models (LLMs) are exposed to multiple examples of input-label pairs, also known as demonstrations, as part of the prompt before performing a task on an unseen example. By thoroughly analyzing these demonstrations, LLMs can uncover potential patterns and effectively adapt to new tasks. Building upon this concept, we propose a **C**ontext-**I**nformed drift-aware method for **S**tock **P**rediction (CISP), which continually adjusts to the latest market styles and offers more accurate predictions. Our proposed method consists of two key parts. Firstly, we introduce a straightforward and efficient technique for designing demonstrations that aggregate current market information, thereby indicating the prevailing stock market style. Secondly, we incorporate a prediction module with dynamic parameters, allowing it to appropriately adjust its model parameters based on the market patterns embedded in the aforementioned demonstrations. Through extensive experiments conducted on real-world stock market datasets, our approach consistently outperforms the most advanced existing methods for stock prediction.

CCS CONCEPTS

• **Applied computing** → **Economics**; • **Computing methodologies** → **Artificial intelligence**.

KEYWORDS

Stock Prediction; Context-Informed; Drift-Aware

*The corresponding authors.

CIKM '23, October 21–25, 2023, Birmingham, United Kingdom

ACM ISBN 979-8-4007-0124-5/23/10…$15.00
https://doi.org/10.1145/3583780.3614886

ACM Reference Format:
Chen-Hui Song, Xi Xiao, Bin Zhang, and Shu-Tao Xia. 2023. Follow the Will of the Market: A Context-Informed Drift-Aware Method for Stock Prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23), October 21–25, 2023, Birmingham, United Kingdom.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3583780.3614886

1 INTRODUCTION

Stock prediction is always a vital area of research in finance that offers valuable insights into market trends and supports informed decision-making for investors. However, the dynamic and complex nature of the market poses significant challenges for researchers, making accurate forecasting of future trends an ongoing and essential pursuit in the finance field.

Deep learning-based methods have gained popularity in recent times as a solution to stock market prediction [16, 19, 20]. Unlike traditional statistical methods, deep learning models excel at capturing complex and non-linear relationships between various market variables that are difficult to model using conventional techniques.

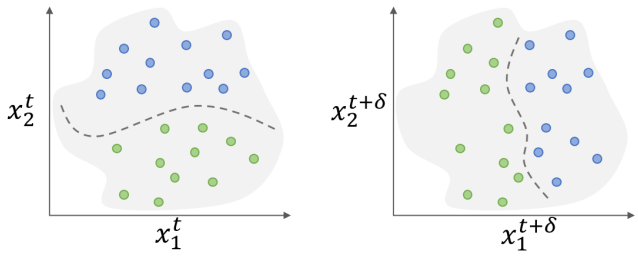

Figure 1: An example illustrating concept drift over time: The green and blue dots represent two distinct classes. The model trained at time t becomes ineffective at time $t + \delta$ due to the evolving decision boundary caused by the changing patterns within the samples over time.

Despite numerous successful attempts [8, 9, 26, 33, 40, 41], stock prediction still encounter persistent challenges. One crucial concern is concept drift [28, 31, 35], which occurs when the data of training set no longer align with recent real-world data due to temporal changes, as illustrated in Figure 1. In the context of stock prediction, most deep learning methods ensure model effectiveness by training

on historical data for an extended period, followed by thorough validation before deployment. This approach introduces a time gap between the training data and the current data. However, the stock market is dynamic and constantly evolving, characterized by fluid relationships between factors driving stock price movements. Factors that were significant in one period may diminish in importance in the next period. For instance, prior to 2017, the A-share market experienced a bull market with increased risk appetite, benefiting small-cap stocks. However, subsequently, as the market shifted to a bearish trend, risk appetite decreased, significantly impacting small-cap stocks. Hence, using outdated historical data may not effectively capture the current market style, leading to potential failures in the model, as the market conditions and relationships it learned may no longer hold true.

Therefore, it is desirable to equip the model with the ability to recognize concept drifts and adapt to emerging market trends. In this regard, the in-context learning method [10, 14, 23, 38], which has gained significant attention in the field of natural language processing (NLP), offers valuable insights. The fundamental concept behind this method is simple yet powerful - analogy. By inputting carefully selected examples (input-label pairs), known as demonstrations, into a large language model (LLM), the model can learn the underlying patterns within these examples and generate predictions that align with the demonstrated patterns. Further experiments [36] indicate that LLMs can even override its semantic priors and adjust to new tasks, when confronted with in-context examples that contradict its prior knowledge. These findings inspire us to consider integrating in-context examples with current market information into stock prediction models. In this approach, stock prediction models can potentially capture the most recent market trends and make necessary adaptations, effectively mitigating the impact of concept drift resulting from the disparity between outdated training data and current data. Nevertheless, this approach also poses two distinct technical challenges, which we will address in this paper.

On the one hand, designing effective demonstrations to assist the model in capturing the current market style is a critical concern. Existing research suggests that there is a factor momentum effect [11] that drives market evolution. While the long-term market style is constantly changing and difficult to predict, in the short term, the market tends to continue its previous style to some extent. Specifically, factors that have been profitable in the recent period may still be profitable in the near future, and conversely, factors that have performed poorly are likely to continue underperforming. In light of these findings, we propose an information aggregation method to construct demonstrations. Firstly, we classify stocks based on their daily performance and then aggregate the features of well-performing stocks and poorly performing stocks separately. These aggregated features can serve as representations of the daily market style on each day. Subsequently, to mitigate the uncertainty caused by market volatility [30], for a given trading day, we utilize a exponential moving average method to further aggregate such market representations from recent trading days, which are then used as demonstrations. Due to the factor momentum effect, these demonstrations are expected to partially reflect the near future market style. It is worth noting that, compared to existing models that only utilize feature information for predictions, demonstrations further explicitly leverage the label information.

On the other hand, a captivating challenge lies in seamlessly integrating demonstrations into the model. Within the realm of NLP tasks, the extraordinary reasoning ability of LLM empowers it to unveil underlying patterns from demonstrations and employ them for inferencing on unseen samples. However, when it comes to stock prediction tasks, we may not have an equally powerful tool like LLM at our disposal. Nonetheless, this is not necessarily a requirement. Our objective is simply to draw inspiration from the concept of in-context learning and devise a suitable network structure that enables the model to adapt to the new market patterns within the above demonstrations. To achieve this, we have introduced a special prediction module with dynamic parameters into our model. Unlike conventional neural networks that possess fixed parameters after training, the parameters of this special prediction module are dynamic and generated through another independent meta network utilizing the aforementioned demonstrations as input. This approach allow us to encode the most recent market patterns within the demonstrations into the prediction module, thereby empowering the model to provide more accurate predictions that align with the current market styles. In contrast, existing models can only learn the market patterns present in the training set and are unable to perceive the latest market changes.

In summary, our proposed **C**ontext-**I**nformed drift-aware **S**tock **P**rediction method (CISP) leverages the idea of in-context learning to address the challenge of concept drift in stock prediction. By incorporating demonstrations and dynamically adjusting model parameters, CISP can effectively capture current market styles and offer more accurate predictions. The main contributions of this paper can be summarized as follows:

- An encoder is pre-trained to extract high-level embedding vectors from the raw data, accompanied by a data preprocessing step that is introduced to address the challenges of handling long sequences and capturing complex high-order dependencies.
- A simple yet efficient information aggregation method is proposed for constructing demonstrations. These demonstrations can serve as effective representations of the most recent market style.
- A context-informed stock prediction module (CISP) is designed to dynamically adjust its parameters based on the demonstrations, enabling the model to provide more accurate predictions aligned with the current market style.
- Extensive experiments are conducted to thoroughly evaluate the performance of our CISP model on three real-world stock market datasets. The results demonstrate that our method significantly outperforms other existing models in stock prediction. Furthermore, additional analysis further validates the effectiveness of our approach.

2 PRELIMINARY

In this section, we present the problem formulation of stock prediction and provide a more precise and rigorous definition of the concept-drift issue that arises within it. Furthermore, we delve into how the concept of in-context learning can be effectively applied to tackle this problem.

2.1 Problem Formulation

The definition of the stock prediction problem can vary depending on the specific investment strategy employed by investors. In this paper, we adopt a widely accepted paradigm in the research field, namely cross-sectional analysis [2, 12]. This approach involves analyzing relevant data at a given moment to predict the future returns of all stocks at a specific point in time. Accurate predictions are expected to provide potential arbitrage opportunities for investors.

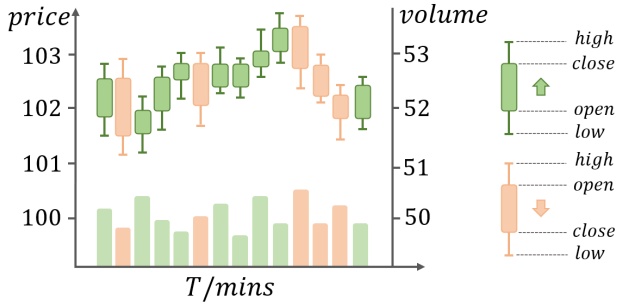

Figure 2: A visualization of minute-level stock data: Each candlestick chart includes the market data of that minute.

Formally, our problem mainly involves learning a function $\hat{y}_t^s = f_{\boldsymbol{\theta}}(\boldsymbol{x}_t^s)$ with parameters $\boldsymbol{\theta}$, which maps the raw data of stocks to their future returns. Specifically, $\boldsymbol{x}_t^s = \{\boldsymbol{x}_t^s[0], \cdots, \boldsymbol{x}_t^s[k-1]\} \in \mathbf{R}^{f \times k}$ represents the minute-level data of stock s on the t-th day. Here, $\boldsymbol{x}_t^s[i]$ denotes the market feature with dimension f for the i-th minute during the trading session on that day. It includes information such as the opening price, closing price, highest price, lowest price, trading volume, turnover, and more. There are k minutes of trading session per day. Figure 2 provides a visual representation of the relevant data. In this paper, our focus lies in predicting the returns of stocks for the next day. Consequently, the labels for the predicted values $\hat{y}_t^s$ are defined as $y_t^s = p_{t+1}^s / p_t^s - 1$, where p_t^s denotes the closing price of stock s on the t-th day.

2.2 Concept Drift

Concept drift refers to a phenomenon wherein the relationship between input and output variables undergoes changes over time, often in undisclosed and inconspicuous manners. It implies that the fundamental concept or the true association between the variables is evolving or shifting, thereby leading to a reduction in the accuracy of predictive models trained on historical data. In formal terms, concept drift can be defined as a modification in the distribution of $p(y|x)$, that is, $p^{t+\delta}(y|x) \neq p^t(y|x)$.

In the above equation, $p^t(y|x)$ represents the conditional probability distribution of the label y given the feature x at time t, while δ denotes a specific time shift. Concept drift often arises due to external factors, which complicates the task of detecting and addressing it. Therefore, a feasible approach is urgently required to alleviate the impact of this issue.

2.3 Context-Informed Drift-Aware Network

In-context learning gained attention in the initial publication of GPT-3 [4], presenting a method to enable language models to learn new tasks with several examples. This technique involves providing the language model with a prompt containing a set of input-label pairs that showcase the desired task. By appending a test input at the conclusion of the prompt, the language model is enabled to make predictions solely by conditioning on the provided prompt.

Specifically, in the conventional prediction paradigm, when we have an input query $\boldsymbol{x}$, the typical approach is to feed $\boldsymbol{x}$ into LLM and obtain the predicted result $\hat{y} = f_{\boldsymbol{M}}(\boldsymbol{x})$. Here, $f_{\boldsymbol{M}}$ represents the pre-trained language model with parameters $\boldsymbol{M}$. However, the in-context learning method differs from this paradigm by introducing a set of demonstrations $\boldsymbol{C}$. This set comprises several demonstration examples in the form of query-label pairs, denoted as $\boldsymbol{C} = \{(\boldsymbol{x}_1, y_1), \cdots, (\boldsymbol{x}_m, y_m)\}$. Then the inference process can be formulated as $\hat{y} = f_{\boldsymbol{M}}(\boldsymbol{x}, \boldsymbol{C})$. This approach explicitly incorporates the features and label information from the demonstrations, thereby enhancing the model's generalization ability. Remarkably, certain experiments have demonstrated that this approach enables the model to transcend the limitations imposed by semantic priors ingrained in the training data. Consequently, the model becomes adaptable to current tasks by discerning new patterns inherent in the demonstrations, even if these patterns contradict the existing semantic priors.

Although we lack powerful tools like LLMs for stock prediction tasks, the concept of utilizing demonstrations to introduce contextual information and aid models in adapting to new tasks holds significant value and is worthy of emulation. In the context of the stock prediction problem mentioned earlier, the phenomenon of concept drift arises when the model's parameters $\boldsymbol{\theta}$ become outdated as market styles evolve, leading to a gradual decline in the effectiveness of the model $f_{\boldsymbol{\theta}}$. However, these changes in market style often stem from external factors and are unforeseeable during training. Thus, it becomes crucial to introduce additional contextual information to enable the model to capture to these changes. Thus, we can design appropriate demonstrations containing current market information to assist the model in adjusting its parameters $\boldsymbol{\theta}$ and adapting to the emerging market styles. In our approach, we have:

$$\hat{y}_t^s = f_{\boldsymbol{\theta}_t}(\boldsymbol{x}_t^s) \text{ and } \boldsymbol{\theta}_t = g(\boldsymbol{c}_t) \tag{1}$$

where $\boldsymbol{\theta}_t$ is is the new parameter generated by an independent network g with the current market information contained in demonstration $\boldsymbol{c}_t$.

3 METHODOLOGY

This section presents our concise and efficient context-informed drift-aware method for stock prediction. To begin with, an encoder is pre-trained to extract high-level embedding vectors from the raw data, alongside a data pre-processing step that is introduced to alleviate the challenges of handling long sequences and capturing complex high-order dependencies. These extracted high-level feature vectors further serve as the foundation for constructing demonstrations that effectively capture and characterize the current market style. Finally, a context-informed stock prediction module (CISP) is designed to dynamically adjust its parameters based on the demonstrations, enabling the model to provide more accurate predictions aligned with the current market style. Figure 3 provides an overview of our method.

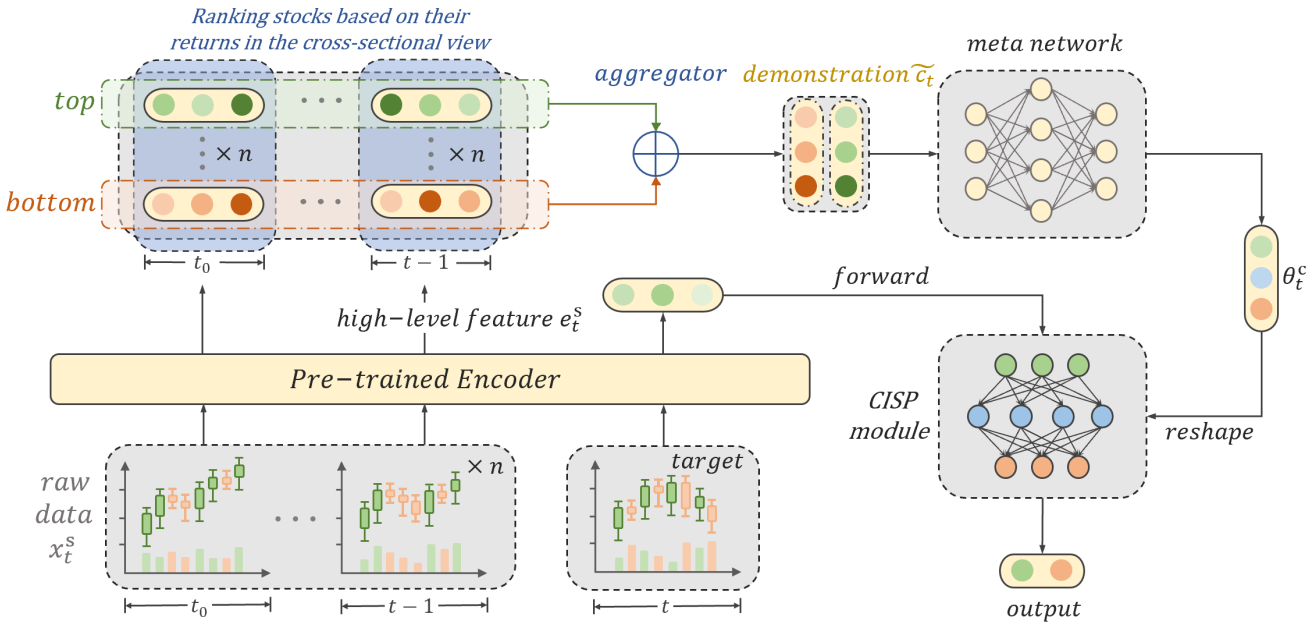

Figure 3: The model overview: Initially, an encoder is pre-trained to convert sequence data into efficient high-level feature vectors for improved representation. Next, using both the feature and label data of stocks, demonstrations are constructed to capture the market style of the day. Finally, a context-informed stock prediction module CISP dynamically generates suitable parameters based on these demonstrations.

3.1 Pre-trained Encoder Module

The raw data of stock market is often noisy [15], and it is presented as long sequences, which poses challenges for the subsequent modules in the model to effectively utilize the data. To address this issue, we employ a pre-training strategy where an encoder is pre-trained to transform the raw data into compact embedding vectors.

3.1.1 Feature Pre-processing. In this paper, we focus on utilizing intraday minute-level data for prediction. This type of data often consists of sequences of over a hundred items per sample. However, excessively long sequences can impede effective learning by the model. Furthermore, even though deep neural networks demonstrate universal approximation ability [29], effectively learning complex high-order dependencies remains a challenge. These dependencies often play a crucial role in predicting future trends in the stock market [1]. Taking these considerations into account, we pre-process the raw data as follows.

Consider the minute-level data $\boldsymbol{x}_t^s \in \mathbf{R}^{f\times k}$ of a specific stock s on a given day t. Each column of $\boldsymbol{x}_t^s$ represents a f-dimensional vector containing market statistics within that minute, such as price, volume, and more. To pre-process the data, we initially divide $\boldsymbol{x}_t^s$ along the time axis into segments of equal length Δ. This process results in a set of data segments $\{\boldsymbol{x}_t^s[0:\Delta-1], \cdots, \boldsymbol{x}_t^s[k-\Delta:k-1]\}$, where $\boldsymbol{x}_t^s[i:i+\Delta-1] \in \mathbf{R}^{f\times\Delta}$ represents the i-th segment. For each data segment, we apply various temporal operators to extract high-order dependencies, which can be uniformly expressed as:

$$operator(x, y, \Delta) \text{ or } operator(x, \Delta) \tag{2}$$

Here, x and y denote input features, and *operator*s represent a class of temporal operators with a window size of Δ. *operator*s in the form of $operator(x, y, \Delta)$ aim to capture the dependencies between different features x and y. For instance, the covariance between price and volume often reflects market trends, thus we can define the operator $covar(closeprice, volume, \Delta)$, which calculates the covariance between the closing price and volume within each window of length Δ. On the other hand, *operator*s in the form of $operator(x, \Delta)$ focus on a single feature. For instance, $std(openprice, \Delta)$ represents the standard deviation of the opening price, indicating the fluctuation in the stock market.

By utilizing a wide range of predefined operators, we can derive multiple high-order statistical quantities and concatenate them into a feature vector. This feature vector effectively captures market trends within each window. Moreover, the pre-processing steps described above significantly reduce the length of the sequence. This reduction in sequence length not only facilitates effective learning but also improves learning efficiency. We represent this transformed data series as $\tilde{\boldsymbol{x}}_{t,\Delta}^s \in \mathbf{R}^{h\times l}$, where h denotes the dimension of high-order features for each window, and $l = k/\Delta$ is the total number of windows. For more detailed information regarding the specific operators employed, please refer to the experiment section 4.1.3.

3.1.2 Sequential Dependency Capture. After the aforementioned pre-processing steps, $\tilde{\boldsymbol{x}}_{t,\Delta}^s$ remains in the form of a sequence, with each column containing the higher-order statistics of the corresponding segment. However, a sequence-based representation is

not optimal for subsequent tasks. Therefore, it is necessary to further transform this sequence into a high-level feature vector. Furthermore, it is also crucial to capture the sequential dependencies among the segments within the feature sequence. For this purpose, we require a encoder module specifically designed to handle such sequential data.

Formally, we denote the aforementioned encoder module as $En(\cdot) : \tilde{\boldsymbol{x}}^s_{t,\Delta} \longmapsto \boldsymbol{e}^s_t$, where $\boldsymbol{e}^s_t = En(\tilde{\boldsymbol{x}}^s_{t,\Delta}) \in \mathbf{R}^o$ represents the high-level feature vector encoded by $En(\cdot)$, and o denotes the embedding dimension. The choice of $En(\cdot)$ is flexible, and in this study, we experiment with two options: GRU [7] and Transformer [32]. For the GRU option, we utilize the output from the last step as the high-level feature vector $\boldsymbol{e}^s_t$. For the Transformer option, we adopt the final output corresponding to the last segment $\tilde{\boldsymbol{x}}^s_{t,\Delta}[l-1]$ from the Transformer encoder.

3.1.3 Pre-training Strategy. To enable the aforementioned encoder $En(\cdot)$ to provide a compact and efficient high-level feature representation of raw data for the subsequent modules, pre-training is necessary. For this purpose, we introduce a multi-layer perceptron (MLP) as a temporary prediction module to generate the final prediction. Formally, the forward process can be expressed as:

$$\hat{y}^s_t = MLP \circ En(\tilde{\boldsymbol{x}}^s_{t,\Delta}) \tag{3}$$

Here, $MLP \circ En$ denotes function composition, where the output of $En(\cdot)$ serves as the input of $MLP(\cdot)$. Subsequently, we can perform supervised training on real market data using mean squared error (MSE) loss, aiming to make the predicted values $\hat{y}^s_t$ closely approximate the true labels y^s_t. Once the training is completed, we discard the $MLP(\cdot)$ module and freeze the parameters of $En(\cdot)$. With the assistance of $En(\cdot)$, the verbose sequential data can be effectively transformed into a more concise high-level vector, facilitating subsequent modules to utilize it more conveniently.

3.2 Demonstration Construction

Equation 1 explains our objective of assisting the model in adapting to the current market style by providing a demonstration $\boldsymbol{c}_t$ with up-to-date market information. Therefore, designing an effective demonstration that accurately reflects the current market state is of paramount importance.

To achieve this, the most straightforward approach is to inform the model about what features indicate good performance of stocks and which ones indicate poor performance. Following this idea, for each trading day t, we rank all stocks based on their future returns y^s_t and select the top and bottom performers (e.g., the top 20% and bottom 20%) to form the sets Λ^+_t and Λ^-_t, respectively. Moreover, we aggregate the features of the selected stocks as follows:

$$\boldsymbol{c}^+_t = \frac{1}{|\Lambda^+_t|} \sum_{s \in \Lambda^+_t} \boldsymbol{e}^s_t \quad \text{and} \quad \boldsymbol{c}^-_t = \frac{1}{|\Lambda^-_t|} \sum_{s \in \Lambda^-_t} \boldsymbol{e}^s_t \tag{4}$$

Here, $\boldsymbol{c}^{+/-}_t \in \mathbf{R}^o$ represents the aggregated feature, and $|\cdot|$ denotes the cardinality of the set, i.e., the number of elements in the set. $\boldsymbol{c}^+_t$ and $\boldsymbol{c}^-_t$ indicate the features of stocks with good and poor performance on a given day t, respectively, thereby roughly characterizing the corresponding market style.

Intuitively, $\boldsymbol{c}^{+/-}_t$ can serve as an ideal demonstration as it fully reflects the types of stocks that are likely to rise and the characteristics of stocks that are expected to decline. However, our objective is to predict stock behavior. Thus, for a given day t, we cannot directly obtain the corresponding market style representation $\boldsymbol{c}^{+/-}_t$, since its computation relies on the label y^s_t, representing the stock's future return. The information available to us is solely the historical data $\boldsymbol{c}^{+/-}_{t'}$, where $t' < t$. Fortunately, recent research has discovered that due to the strong persistence of investor sentiment, stock factors (e.g., turnover rate, size, value, etc.) exhibit momentum effects [11]. In other words, although the market style changes over the long term, factors that have been profitable in a recent period are likely to continue being profitable in near future, while those that have been losing may continue to decline. These findings allow us to approximate $\boldsymbol{c}^{+/-}_t$ using their values from recent times. To some extent, the aforementioned high-level features $\boldsymbol{e}^s_t$ can also be seen as a special type of factor as they also describe the market's state. Therefore, momentum effects are applicable to them as well and we can approximate $\boldsymbol{c}^{+/-}_t$ by aggregating historical data. Specifically, we employ the exponential moving average (EMA) method to achieve this, with the following details:

$$\begin{aligned} \tilde{\boldsymbol{c}}^+_t &= \alpha \boldsymbol{c}^+_{t-1} + (1-\alpha)\tilde{\boldsymbol{c}}^+_{t-1} \text{ with } \tilde{\boldsymbol{c}}^+_0 = \mathbf{0} \\ \tilde{\boldsymbol{c}}^-_t &= \alpha \boldsymbol{c}^-_{t-1} + (1-\alpha)\tilde{\boldsymbol{c}}^-_{t-1} \text{ with } \tilde{\boldsymbol{c}}^-_0 = \mathbf{0} \end{aligned} \tag{5}$$

Here, $\tilde{\boldsymbol{c}}^{+/-}_t$ serves as an approximation of $\boldsymbol{c}^{+/-}_t$, and α represents an adjustable parameter. The usage of EMA offers two distinct advantages. Firstly, the stock market is prone to volatility and randomness, making it risky to rely solely on the most recent data. By calculating the average, we can smoothen out fluctuations to a certain extent and alleviate the impact of volatility. Secondly, the similarity of market style resulting from momentum effects only persists for a limited period. Over the long term, this similarity may cease to hold true. Consequently, EMA assigns greater importance to recent data and iteratively updates every day, enabling the approximation $\tilde{\boldsymbol{c}}^{+/-}_t$ to keep up with market changes.

Lastly, we aggregate $\tilde{\boldsymbol{c}}^+_t$ and $\tilde{\boldsymbol{c}}^-_t$ as follows:

$$\tilde{\boldsymbol{c}}_t = \tilde{\boldsymbol{c}}^+_t - \tilde{\boldsymbol{c}}^-_t \tag{6}$$

Intuitively, $\tilde{\boldsymbol{c}}_t$ represents the differences in features between stocks performing well and those performing poorly in the current market. Thus, it can serve as a demonstration to assist the subsequent prediction module in adapting to the current market style and making more accurate predictions. Unlike existing stock prediction models that rely solely on feature information for prediction, the aforementioned demonstrations explicitly leverage label information. Meanwhile, the aggregation method based on EMA is also highly efficient, requiring minimal computation and storage space.

3.3 Context-Informed Stock Prediction Module

To fully leverage the valuable information embedded in the aforementioned demonstrations and enhance the predictive performance of our model, it is imperative to develop a tailored network architecture that can seamlessly integrate this information. In the realm of NLP, LLMs possess remarkable reasoning abilities, enabling them to directly capture underlying patterns in demonstrations and adapt to new tasks. However, such powerful tools are currently absent

for stock prediction. Fortunately, recent advancements in dynamic parameter neural networks [39] within the field of recommendation systems have provided us with a viable approach to implementing a context-informed adaptive network. However, unlike existing methodologies that rely on a self-wise approach for adjusting network parameters based on its input features, we extend this approach by incorporating external information, demonstrations with market information, to aid the model in adapting to the current market style.

In essence, our approach revolves around training a meta network, denoted as $g(\cdot)$, to dynamically generate parameters $\boldsymbol{\theta}_t$ for the model $f_{\boldsymbol{\theta}_t}$, thereby enabling $f_{\boldsymbol{\theta}_t}$ to gain more appropriate parameters that yield more accurate predictions. Specifically, given that the encoder part of our model is pre-trained and its parameters $\boldsymbol{\theta}^e \in \boldsymbol{\theta}_t$ are fixed after pre-training, our focus lies in designing a context-informed stock prediction module (CISP) to deliver the final prediction. The parameters $\boldsymbol{\theta}_t^c \in \boldsymbol{\theta}_t$ for CISP are generated by the meta network g using the aforementioned demonstration $\tilde{\boldsymbol{c}}_t$. Consequently, our model can be expressed as follows:

$$\hat{y}_t^s = CISP_{\boldsymbol{\theta}_t^c} \circ En_{\boldsymbol{\theta}^e}(\tilde{\boldsymbol{x}}_t^s) \tag{7}$$

where only parameters of CISP and g are further updated in this stage.

For the sake of efficiency and convenience, both CISP and g adopt MLP structures. Specifically, for the i-th layer of CISP and the demonstration $\tilde{\boldsymbol{c}}_t$, a corresponding meta network g_i is trained to generate the weights $\boldsymbol{W}_i^t$ for that layer. This can be expressed as:

$$\boldsymbol{W}_i^t = reshape \circ g_i(\tilde{\boldsymbol{c}}_t) \in \mathbf{R}^{M_i \times N_i} \tag{8}$$

Here, M_i/N_i represents the input/output dimensions of the i-th layer, and the operator $reshape$ transforms the vectors generated by g_i into a matrix format. With the daily fluctuations in the demonstration $\tilde{\boldsymbol{c}}_t$, the weight $\boldsymbol{W}_i^t$ of each layer in CISP also undergo daily changes. Notably, for each layer i of CISP, there is a distinct meta network g_i to generate its weights. The forward process of CISP can then be formulated as:

$$\boldsymbol{h}_t^s[i+1] = \sigma(\boldsymbol{W}_i^t \boldsymbol{h}_t^s[i]) \text{ with } \boldsymbol{h}_t^s[0] = \boldsymbol{e}_t^s \tag{9}$$

where σ denotes the activation function and $\boldsymbol{h}_s^t[i]$ corresponds to the i-th hidden feature of CISP. The hidden feature of the last layer will be projected as a scalar without passing through an activation layer, which will serve as the final prediction result.

In practical implementation, to address the issues of time and memory inefficiency, we often decompose $\boldsymbol{W}_i^t$ into a low-rank matrix, typically referred to as:

$$\boldsymbol{W}_i^t = \boldsymbol{U}_i \boldsymbol{I}_i^t \boldsymbol{V}_i \tag{10}$$

Here, $\boldsymbol{U}_i \in \mathbf{R}^{M_i \times K_i}$ and $\boldsymbol{V}_i \in \mathbf{R}^{K_i \times N_i}$ represent the shared parameters ($K_i \ll min(M_i, N_i)$, except for the last layer) for the i-th layer, while only $\boldsymbol{I}_i^t = g_i(\tilde{\boldsymbol{c}}_t) \in \mathbf{R}^{K_i \times K_i}$ is generated by g_i. This approach not only reduces the parameter count but also enables the shared parameters $\boldsymbol{U}_i, \boldsymbol{V}_i$ to extract common features, thereby enhancing the efficiency and effectiveness of the inferential process.

3.4 Training Strategy

The training of the entire model can be divided into two parts. Firstly, we train the encoder part $En(\cdot)$, as discussed in section 3.1.3. Once the pre-training is completed, we discard the temporary prediction module and freeze the parameters of $En(\cdot)$. Then, leveraging the high-level features $\boldsymbol{e}_t^s = En(\tilde{\boldsymbol{x}}_t^s)$, we construct demonstrations $\tilde{\boldsymbol{c}}_t$ to estimate the market style for each trading day. These demonstrations further guide the CISP module in generating appropriate parameters for the final prediction. Both stages of training can be formulated as optimizing the following loss function:

$$\mathcal{L} = \frac{1}{|S|} \frac{1}{|D|} \sum_{s \in S} \sum_{t \in D} \|\hat{y}_t^s - y_t^s\|^2 \tag{11}$$

Here, S represents the set of stocks, and D includes the trading days in training set. Additionally, weight decay is introduced during optimization to mitigate overfitting. Notably, in the second phase, only the parameters of $\boldsymbol{U}_i$ and $\boldsymbol{V}_i$, and g_i are optimized, while $\boldsymbol{I}_i^t$ is generated by g_i.

4 EXPERIMENTS

In this section, we validate the effectiveness of our method through extensive experiments on real market data. Here we focus on predicting daily stock returns based on minute-level market data within a trading day. We evaluate our approach on three representative datasets and show that it outperforms existing methods significantly.

4.1 Experiment Setups

4.1.1 Dataset. To evaluate the effectiveness of our approach, we conducted experiments using real-world stock market data, specifically intraday minute-level data for stocks included in three prominent A-share indexes of China: **CSI300**, **CSI500**, and **CSI800**. Notably, these datasets are highly representative as CSI300 represents large-cap stocks, CSI500 primarily consists of mid-cap and small-cap stocks, and CSI800 includes both. Therefore, our experiments provide a comprehensive and unbiased assessment of our model's performance in real-world scenarios. The intraday minute-level data comprises the opening, closing, highest, and lowest prices, as well as the turnover, trading volume, and number of transactions for each stock at every minute of the trading day. Additionally, we calculated the minute-by-minute change rates of these features as supplementary information, resulting in a total of 14 features. The predictive labels are the stocks' return rates of the following day, which are normalized using the Z-Score method. Finally, we divided the dataset into training (2015/01/01 - 2018/12/31), validation (2019/01/01 - 2019/12/31), and testing (2020/01/01 - 2022/07/01) sets based on the time sequence.

4.1.2 Evaluation Metrics. To make a comprehensive comparison between our model and existing methods, we utilize several evaluation metrics. These metrics reflect the models' predictive performance from various angles, allowing us to gain a holistic understanding of their strengths and weaknesses.

- **$\overline{\text{IC}}$ and $\overline{\text{RankIC}}$:** IC_i represents the Pearson correlation coefficient between the returns of all stocks and their corresponding predicted values on day i. $RankIC_i$ refers to the Spearman correlation coefficient on day i. $\overline{IC}$ and $\overline{RankIC}$ denote the average IC and RankIC, respectively, calculated over all the trading days.

- **ICIR and RankIR:** Both the ICIR and the RankIR reflect the stability of the predictive signal, and they can be calculated as the ratio of the mean and standard deviation of daily IC and daily RankIC over a given period of time.
- **Long-side and Short-side Return:** These metrics provide insights into the profitability of the model. Specifically, we categorize stocks each day into the top 10% and bottom 10% based on their predicted values. We then calculate the average return for the following day for these two groups of stocks, representing the long-side return ($\overline{Return^+}$) and short-side return ($\overline{Return^-}$), respectively.
- **Long-Short Average Return:** It is the average of long-short return, namely, $\overline{Return} = \frac{1}{|T|}\sum_{i\in T} Return_i$, where the long-short return $Return_i$ is defined as $(Return_i^+ - Return_i^-)/2$.
- **Long-Short Average Sharpe:** The Sharpe ratio measures the model's ability to generate stable profits and it can be calculated as the the ratio of the mean and standard deviation of long-short Return. That is, $Sharpe = \overline{Return}/std(Return)$.

4.1.3 Implementation Details. Regarding the encoder module $En(\cdot)$, we adopted two models, namely GRU and Transformer, to capture sequence dependencies. The GRU is a single-layer structure with a hidden layer dimension of 128, while the Transformer consists of two layers with a feature dimension of 128. For the CISP module, we used an MLP architecture with hidden layers of $[64, 64]$ and employed $tanh$ as the activation function. Each layer's parameters in the CISP module are generated by an independent meta network g_i (for the i-th layer), which also has an MLP architecture. The rank K_i is uniformly set to 16. We also provide a brief introduction to the temporal operators introduced in Section 3.1.1. These operators act on a time window of size Δ. The operators in the form of $operator(x, y, \Delta)$ express the interaction between feature x and feature y, including correlation, coskewness, cokurtosis, and more. These operators help us capture complex relationships between features and identify potential dependencies that may not be obvious at first glance. On the other hand, the operator in the form of $operator(x, \Delta)$ represents the individual characteristics of feature x, such as min, max, mean, standard deviation, skewness, kurtosis, z-score normalization, and so on. Additionally, choosing an appropriate Δ is crucial. A larger Δ implies coarse-grained information, helping alleviate market noise but potentially losing some details. Conversely, a smaller Δ provides fine-grained information, including more details but becoming more susceptible to market noise interference. To strike a balance, Δ is set to 15.

4.1.4 Comparison Methods. We evaluated the performance of our model by comparing it with the following models:

- **MLP** is a simple yet widely used benchmark model.
- **XGBoost** [5] and **LGBM** [18] are typical representatives of tree-based models, and many studies have shown that they deliver excellent and stable performance in tabular data tasks.
- Recurrent networks are commonly used for sequential tasks. Among them, **GRU** [7] is a stable and efficient implementation of recurrent networks. **ALSTM** [34], on the other hand, introduces an attention mechanism based on LSTM. Additionally, **SFM** [43] utilizes Discrete Fourier Transform to discover patterns in the data.
- **TCN** [3] leverages convolutional operations to uncover potential patterns in temporal sequences.
- **Transfomer (Trans.)** [32] benefits from the multi-head attention mechanism, allowing it to efficiently capture dependencies between sequences. It has demonstrated excellent performance in many fields.

4.2 Experimental Results

Firstly, we explore the impact of data pre-processing on model performance. For this purpose, we designed a comparative experiment where two different GRU models are trained using raw feature $\boldsymbol{x}_t^s$ and pre-processed feature $\tilde{\boldsymbol{x}}_{t,\Delta}^s$ as inputs, respectively. The results, shown in Figure 4, indicate that the model's performance (measured by the $\overline{IC}$ value) is significantly better when using pre-processed data compared to the use of the raw data across all three datasets. This suggests that data pre-processing greatly alleviates the challenges faced by deep learning models in handling long sequences and capturing high-order features.

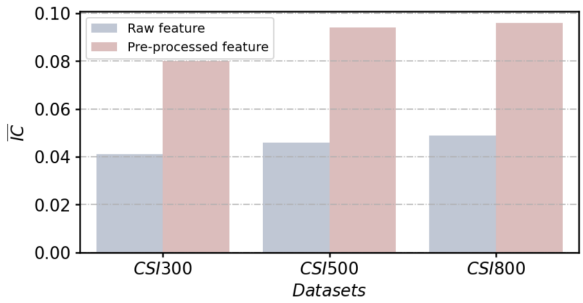

Figure 4: The impact of feature pre-processing on the performance of the GRU model.

Then we proceed to evaluate the predictive performance of each model. Given the significant impact of data pre-processing on model performance, all models utilize pre-processed data as inputs in subsequent experiments to ensure a fair comparison. Table 1 showcases the experimental results on the CSI300, CSI500, and CSI800 datasets, derived from real data in the A-share stock market. We observe that simple tree-based models, such as XGBoost and LGBM, achieve favorable performance across multiple metrics, affirming their reliability as baselines for tabular data, in line with previous findings [13]. Furthermore, meticulously designed deep models exhibit varying degrees of improvement compared to the simple MLP, with GRU and Transformer demonstrating particularly promising outcomes. Consequently, we integrate GRU and Transformer as encoders $En(\cdot)$ into our proposed CISP module. To ensure result reliability and eliminate experimental variability, the GRU+CSIP model shares GRU parameters with the standalone GRU model for each dataset, and the same principle applies to the Transformer. The results clearly demonstrate the notable advantages of incorporating the CISP module. By introducing CISP, we effectively enhance various key metrics, encompassing predictive ability ($\overline{IC}$, $\overline{RankIC}$),

Table 1: Quantitative comparisons among different methods. The bold numbers represent the optimal results, while the numbers with underscores represent suboptimal results. $\overline{Return^+}$, $\overline{Return^-}$ and $\overline{Return}$ are measured in percentage (%).

Dateset	Method	Metrics							
		$\overline{IC}\uparrow$	$ICIR\uparrow$	$\overline{RankIC}\uparrow$	$RankIR\uparrow$	$\overline{Return^+}\uparrow$	$\overline{Return^-}\downarrow$	$\overline{Return}\uparrow$	$Sharpe\uparrow$
CSI 300	MLP	0.057	0.514	0.078	0.715	0.278	-0.195	0.237	0.467
	XGBoost	0.082	0.700	0.078	0.715	0.322	-0.291	0.306	0.560
	LGBM	0.083	0.693	0.078	0.717	0.335	-0.296	0.315	0.559
	TCN	0.073	0.602	0.080	0.652	0.315	-0.296	0.306	0.543
	SFM	0.066	0.621	0.076	0.743	0.276	-0.268	0.272	0.540
	ALSTM	0.072	0.623	0.077	0.700	0.303	-0.304	0.303	0.553
	GRU	0.080	0.670	0.081	0.661	0.297	**-0.348**	0.323	0.597
	Transfomer	0.085	0.713	0.092	0.864	0.352	-0.315	0.333	0.619
	GRU+CISP	0.086	0.768	0.089	0.843	0.356	-0.323	0.339	0.652
	Trans.+CISP	**0.093**	**0.841**	**0.094**	**0.878**	**0.365**	-0.328	**0.347**	**0.676**
CSI 500	MLP	0.066	0.640	0.083	0.811	0.325	-0.256	0.290	0.581
	XGBoost	0.094	0.922	0.099	1.141	0.399	-0.350	0.374	0.797
	LGBM	0.093	0.768	0.093	1.112	0.375	-0.336	0.356	0.772
	TCN	0.084	0.860	0.094	1.065	0.378	-0.369	0.374	0.779
	SFM	0.073	0.744	0.087	1.018	0.321	-0.339	0.330	0.676
	ALSTM	0.097	1.063	0.096	1.121	0.406	-0.395	0.401	0.834
	GRU	0.094	0.862	0.098	1.043	0.344	-0.402	0.373	0.746
	Transfomer	0.099	0.887	0.101	1.128	0.415	-0.369	0.392	0.737
	GRU+CISP	0.105	1.075	**0.106**	**1.165**	**0.437**	**-0.422**	**0.429**	0.887
	Trans.+CISP	**0.108**	**1.116**	0.101	1.151	0.430	-0.414	0.422	**0.896**
CSI 800	MLP	0.068	0.747	0.083	0.811	0.317	-0.261	0.289	0.667
	XGBoost	0.098	1.076	0.097	1.135	0.385	-0.354	0.370	0.818
	LGBM	0.094	1.080	0.085	1.100	0.360	-0.332	0.346	0.853
	TCN	0.088	1.012	0.092	1.030	0.326	-0.353	0.339	0.838
	SFM	0.086	1.026	0.086	1.115	0.339	-0.343	0.341	0.842
	ALSTM	0.097	1.031	0.101	1.074	0.367	-0.374	0.371	0.859
	GRU	0.096	1.061	0.100	1.163	0.404	-0.408	0.406	0.926
	Transfomer	0.103	1.132	0.101	1.147	0.406	-0.403	0.404	0.922
	GRU+CISP	0.104	1.120	**0.107**	**1.231**	0.432	**-0.409**	0.420	0.941
	Trans.+CISP	**0.111**	**1.219**	0.102	1.169	**0.448**	-0.404	**0.426**	**0.974**

profitability ($\overline{Return^+}$, $\overline{Return^-}$, $\overline{Return}$), and prediction stability ($ICIR$, $\overline{RankIC}$, $Sharpe$).

4.3 Further Analysis

In this section, we conduct further research on the CISP module to confirm its effectiveness. For this purpose, we select the GRU+CISP model as a representative and perform a series of relevant experiments on the CSI500 dataset.

Firstly, we need to confirm whether CISP is truly capable of adjusting its parameters based on demonstrations, rather than disregarding demonstrations and still learning a fixed mapping from input $\tilde{\boldsymbol{x}}_t^s$ to label y_t^s. To this end, we attempt to introduce noise to the demonstration vector and observe the change in model performance. Specifically, we modify the demonstration vector using the following equation:

$$\tilde{\boldsymbol{c}}_t' = \tilde{\boldsymbol{c}}_t + \gamma \boldsymbol{N}(0,1) \tag{12}$$

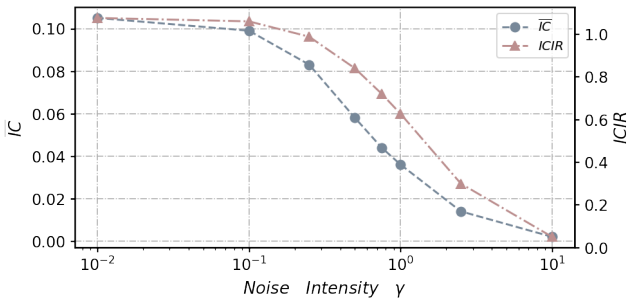

Figure 5: The impact of the noise intensity γ added to the demonstration vector on $\overline{IC}$ and $ICIR$.

Here, $\tilde{\boldsymbol{c}}_t'$ represents the demonstration vector with added noise, and $\boldsymbol{N}(0,1)$ denotes a standard normal distribution vector with the

Table 2: The model performance varies for the input demonstration $\tilde{c}_{t-d}$ with different time shifts d.

d	$\overline{IC}\uparrow$	$ICIR\uparrow$	$\overline{Return}\uparrow$	$Sharpe\uparrow$
0	0.105	1.075	0.429	0.887
100	0.101	1.015	0.410	0.855
200	0.100	1.009	0.405	0.826
300	0.099	1.003	0.404	0.834
400	0.097	0.987	0.396	0.815
500	0.096	0.993	0.391	0.822

same shape as $\tilde{\boldsymbol{c}}_t$. The intensity of the noise is controlled by the parameter $\gamma > 0$. Figure 5 clearly demonstrates that as the noise intensity increases, the model performance continuously deteriorates, indicating the importance of the demonstration for the model.

Furthermore, our objective is to confirm the presence of concept drift in the stock market and determine if the performance improvement achieved by CISP can be attributed, at least in part, to mitigate this concept drift. To accomplish this, we conduct a comparative analysis on market styles across different time periods, utilizing cosine similarity as a quantitative measure. Specifically, we take

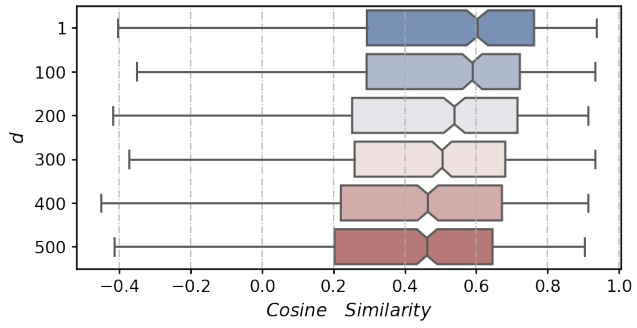

Figure 6: The distribution of element values in $\boldsymbol{sim}(d)$ for different time shift values d.

$\boldsymbol{c}_t^+$ as an example, which represents the style of well-performing stocks for each trading day t. For each trading day t in the test set $\boldsymbol{T}$, we calculate the cosine similarity between $\boldsymbol{c}_t^+$ and $\boldsymbol{c}_{t-d}^+$, where $\boldsymbol{c}_{t-d}^+$ denotes the market style d days ago. The resulting values are collected in the set $\boldsymbol{sim}(d)$, defined as follows:

$$\boldsymbol{sim}(d) = \left\{ \frac{\boldsymbol{c}_t^+ \cdot \boldsymbol{c}_{t-d}^+}{\|\boldsymbol{c}_t^+\| \|\boldsymbol{c}_{t-d}^+\|}, t \in \boldsymbol{T} \right\} \tag{13}$$

We then examine the distribution of element values in $\boldsymbol{sim}(d)$ for different time shifts d and present the results in Figure 6. The results reveal that as the time shift d increases, the overall values of the elements in $\boldsymbol{sim}(d)$ decrease. This indicates that market styles gradually change over time, confirming the presence of concept drift.

However, we observe that market styles exhibit higher similarity between closer time periods, which supports the existence of factor momentum. In further comparative experiments, for the trained GRU+CISP model, we substitute $\tilde{\boldsymbol{c}}_{t-d}$ for $\tilde{\boldsymbol{c}}_t$ as the input demonstration to CISP. The results in Table 2 demonstrate that utilizing demonstrations that represent more recent market styles leads to better model performance. By combining the aforementioned experiments on market style similarity, we further confirm that the improvement in model performance can be attributed to the CISP module's ability to effectively track and adapt to the most recent market styles. Therefore, this is a context-informed drift-aware model.

5 RELATED WORKS

Stock Prediction Methods: Stock prediction is always a significant concern in finance. Some approaches focus on improving the model's structure [25, 34, 43] to better capture inherent patterns, while some others utilize additional alternative data [6, 17, 22, 24, 27, 37] like relationships between stocks, textual data, and social media data . There are also efforts dedicated to training and combining models [21, 42] in different ways. But without exception, these methods are all based on training models using a substantial amount of historical data, followed by a period of validation before deployment. This limitation hinders the models from adapting to the most recent market patterns.

Adaptive Parameter Generation Network: On the technical side, recent developments in the field of recommendation systems, specifically the Adaptive Parameter Generation Network [39], have inspired our work. It enhances model performance on CTR tasks by generating tailored parameters for each sample. However, existing research primarily focuses on parameter generation based solely on sample's inherent features (e.g., the input features). In contrast, our work expands this approach by explicitly incorporating external information to aid the model in adapting to new market styles.

6 CONCLUSION

In this work, we present a novel solution for mitigating concept drift in stock prediction. We address the challenge by introducing a context-informed model inspired by In-Context learning. Firstly, we propose a data pre-processing technique and a pre-trained encoder to handle complex stock data. Subsequently, we develop an information aggregation method to construct demonstrations that accurately represent recent market trends. By dynamically adjusting its parameters based on these demonstrations, our CISP model has provided more precise predictions in line with the current market style. Extensive experiments conducted on real-world datasets demonstrate the superiority of our approach over existing models.

ACKNOWLEDGMENTS

This work is supported in part by the National Natural Science Foundation of China (61972219 and 62171248), the Science and Technology Program of Shenzhen (JCYJ20220818101012025), the Research and Development Program of Shenzhen (JCYJ20190813174403598), the Overseas Research Cooperation Fund of Tsinghua Shenzhen International Graduate School (HW2021013), and the Key Project of Peng Cheng Laboratory (PCL2021A07 and PCL2023AS4-1).

REFERENCES

[1] Diego Amaya, Peter Christoffersen, Kris Jacobs, and Aurelio Vasquez. 2011. *Do Realized Skewness and Kurtosis Predict the Cross-Section of Equity Returns?* CREATES Research Papers 2011-44. Department of Economics and Business Economics, Aarhus University.

[2] Hossein Asgharian and Björn Hansson. 2002. *Cross Sectional Analysis of the Swedish Stock Market.* Technical Report. Working Paper.

[3] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271* (2018).

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Tianqi Chen, Tong He, Michael Benesty, Vadim Khotilovich, Yuan Tang, Hyunsu Cho, Kailong Chen, Rory Mitchell, Ignacio Cano, Tianyi Zhou, et al. 2015. Xgboost: extreme gradient boosting. *R package version 0.4-2* 1, 4 (2015), 1–4.

[6] Wei Chen, Manrui Jiang, Wei-Guo Zhang, and Zhensong Chen. 2021. A novel graph convolutional feature based convolutional neural network for stock trend prediction. *Information Sciences* 556 (2021), 67–94.

[7] Rahul Dey and Fathi M Salem. 2017. Gate-variants of gated recurrent unit (GRU) neural networks. In *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*. IEEE, 1597–1600.

[8] Daizong Ding, Mi Zhang, Xudong Pan, Min Yang, and Xiangnan He. 2019. Modeling extreme events in time series prediction. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1114–1122.

[9] Qianggang Ding, Sifan Wu, Hao Sun, Jiadong Guo, and Jian Guo. 2020. Hierarchical Multi-Scale Gaussian Transformer for Stock Movement Prediction.. In *IJCAI*. 4640–4646.

[10] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. 2022. A Survey for In-context Learning. *arXiv preprint arXiv:2301.00234* (2022).

[11] Sina Ehsani and Juhani T Linnainmaa. 2022. Factor momentum and the momentum factor. *The Journal of Finance* 77, 3 (2022), 1877–1919.

[12] Robin M Greenwood. 2005. A cross sectional analysis of the excess comovement of stock returns. *Available at SSRN 698162* (2005).

[13] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on tabular data? *arXiv preprint arXiv:2207.08815* (2022).

[14] Michael Hahn and Navin Goyal. 2023. A theory of emergent in-context learning as implicit structure induction. *arXiv preprint arXiv:2303.07971* (2023).

[15] Peter R Hansen and Asger Lunde. 2006. Realized variance and market microstructure noise. *Journal of Business & Economic Statistics* 24, 2 (2006), 127–161.

[16] Weiwei Jiang. 2021. Applications of deep learning in stock market prediction: recent progress. *Expert Systems with Applications* 184 (2021), 115537.

[17] Sneh Kalra and Jay Shankar Prasad. 2019. Efficacy of news sentiment for stock market prediction. In *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon)*. IEEE, 491–496.

[18] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).

[19] Deepak Kumar, Pradeepta Kumar Sarangi, and Rajit Verma. 2022. A systematic review of stock market prediction using machine learning and statistical techniques. *Materials Today: Proceedings* 49 (2022), 3187–3191.

[20] Mahinda Mailagaha Kumbure, Christoph Lohrmann, Pasi Luukka, and Jari Porras. 2022. Machine learning techniques and data for stock market forecasting: A literature review. *Expert Systems with Applications* (2022), 116659.

[21] Jiazheng Li, Linyi Yang, Barry Smyth, and Ruihai Dong. 2020. Maec: A multimodal aligned earnings conference call dataset for financial risk prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 3063–3070.

[22] Wei Li, Ruihan Bao, Keiko Harimoto, Deli Chen, Jingjing Xu, and Qi Su. 2021. Modeling the stock relation with graph network for overnight stock movement prediction. In *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*. 4541–4547.

[23] Xiaonan Li and Xipeng Qiu. 2023. Finding supporting examples for in-context learning. *arXiv preprint arXiv:2302.13539* (2023).

[24] Yelin Li, Hui Bu, Jiahong Li, and Junjie Wu. 2020. The role of text-extracted investor sentiment in Chinese stock price prediction with the enhancement of deep learning. *International Journal of Forecasting* 36, 4 (2020), 1541–1562.

[25] Hengxu Lin, Dong Zhou, Weiqing Liu, and Jiang Bian. 2021. Learning multiple stock trading patterns with temporal routing adaptor and optimal transport. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1017–1026.

[26] Guang Liu, Yuzhao Mao, Qi Sun, Hailong Huang, Weiguo Gao, Xuan Li, Jianping Shen, Ruifan Li, and Xiaojie Wang. 2020. Multi-scale Two-way Deep Neural Network for Stock Trend Prediction.. In *IJCAI*. 4555–4561.

[27] Jiawei Long, Zhaopeng Chen, Weibing He, Taiyu Wu, and Jiangtao Ren. 2020. An integrated framework of deep learning and knowledge graph for prediction of stock price trend: An application in Chinese stock exchange market. *Applied Soft Computing* 91 (2020), 106205.

[28] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering* 31, 12 (2018), 2346–2363.

[29] Franco Scarselli and Ah Chung Tsoi. 1998. Universal approximation using feedforward neural networks: A survey of some existing methods, and some new results. *Neural networks* 11, 1 (1998), 15–37.

[30] G William Schwert. 1990. Stock market volatility. *Financial analysts journal* 46, 3 (1990), 23–34.

[31] Alexey Tsymbal. 2004. The problem of concept drift: definitions and related work. *Computer Science Department, Trinity College Dublin* 106, 2 (2004), 58.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[33] Heyuan Wang, Tengjiao Wang, and Yi Li. 2020. Incorporating expert-based investment opinion signals in stock prediction: A deep learning framework. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 971–978.

[34] Qi Wang and Yongsheng Hao. 2020. ALSTM: An attention-based long short-term memory framework for knowledge base reasoning. *Neurocomputing* 399 (2020), 342–351.

[35] Shenghui Wang, Stefan Schlobach, and Michel Klein. 2011. Concept drift and how to identify it. *Journal of Web Semantics* 9, 3 (2011), 247–265.

[36] Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, et al. 2023. Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846* (2023).

[37] Jimmy Ming-Tai Wu, Zhongcui Li, Norbert Herencsar, Bay Vo, and Jerry Chun-Wei Lin. 2021. A graph-based CNN-LSTM stock price prediction algorithm with leading indicators. *Multimedia Systems* (2021), 1–20.

[38] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2021. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080* (2021).

[39] Bencheng Yan, Pengjie Wang, Kai Zhang, Feng Li, Hongbo Deng, Jian Xu, and Bo Zheng. 2022. Apg: Adaptive parameter generation network for click-through rate prediction. *Advances in Neural Information Processing Systems* 35 (2022), 24740–24752.

[40] Linyi Yang, Jiazheng Li, Ruihai Dong, Yue Zhang, and Barry Smyth. 2022. NumHTML: Numeric-Oriented Hierarchical Transformer Model for Multi-task Financial Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 11604–11612.

[41] Jaemin Yoo, Yejun Soun, Yong-chan Park, and U Kang. 2021. Accurate multivariate stock movement prediction via data-axis transformer with multi-level contexts. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2037–2045.

[42] Chuheng Zhang, Yuanqi Li, Xi Chen, Yifei Jin, Pingzhong Tang, and Jian Li. 2020. Doubleensemble: A new ensemble method based on sample reweighting and feature selection for financial data analysis. In *2020 IEEE International Conference on Data Mining (ICDM)*. IEEE, 781–790.

[43] Liheng Zhang, Charu Aggarwal, and Guo-Jun Qi. 2017. Stock price prediction via discovering multi-frequency trading patterns. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2141–2149.